Applied A.I. Solutions Development Program

FULL STACK DATA SCIENCE SYSTEMS

Smart AI Photo Cleaner APP

Final Presentation

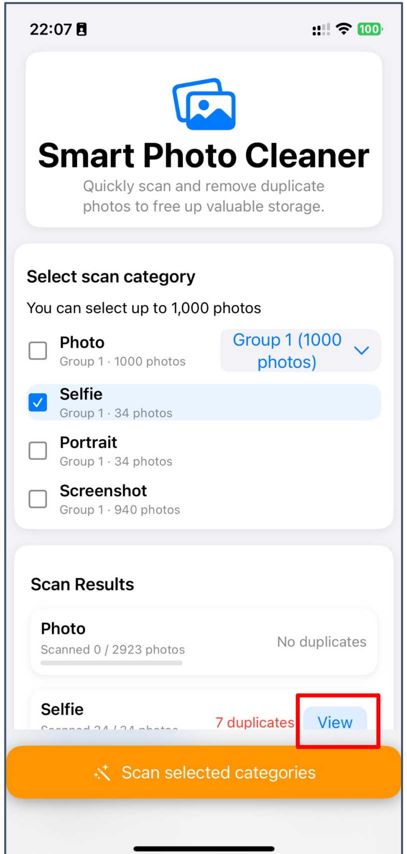
Group 13
Chia-Wei Chang (101570243)

Supervised by: Dr. Vejey Gandyer

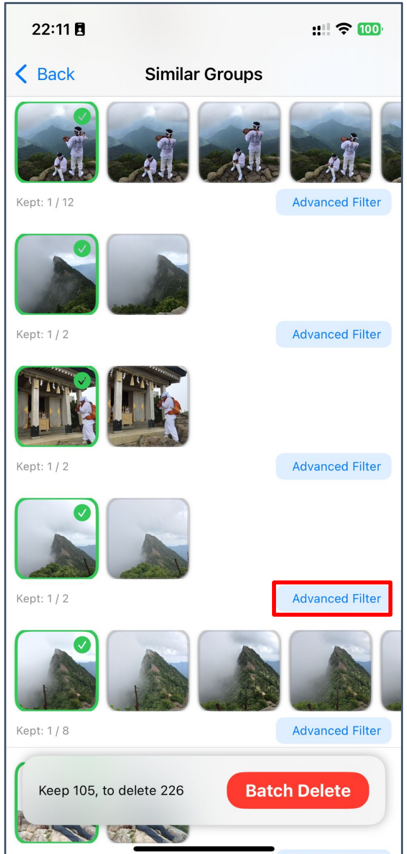
Agenda

- Problem Statement
- Past Available Projects
- ML Canvas
- Model Benchmarking
- Model Deployment
- Demo
- Q&A

Smart AI Photo Cleaner APP Demo

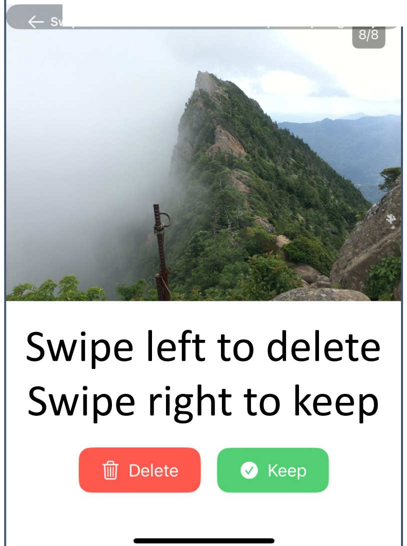


Click View



Click Advanced Filter

APP Download Link



Problem Statement

- Modern phones take many similar high-resolution photos, quickly consuming storage.
- Deleting these manually is a pain.

Source Link

Source Link

Many people find that their phone storage is mostly taken up by photos. This is especially true because modern phones take high-resolution pictures and support burst shot mode. With burst shots, users often take many similar or duplicate photos at once, which makes organizing them difficult. As a result, these large photo libraries end up using a lot of valuable storage space.

Why This Problem

- I noticed I often run out of storage on my phone because of too many similar and duplicate photos.
- My friends also have this problem and are even thinking about buying phones with more storage.
- Then I realized: maybe AI can help clean up our photos and save space and no need to upgrade devices!

Smart AI Photo Cleaner APP

Past Available Projects

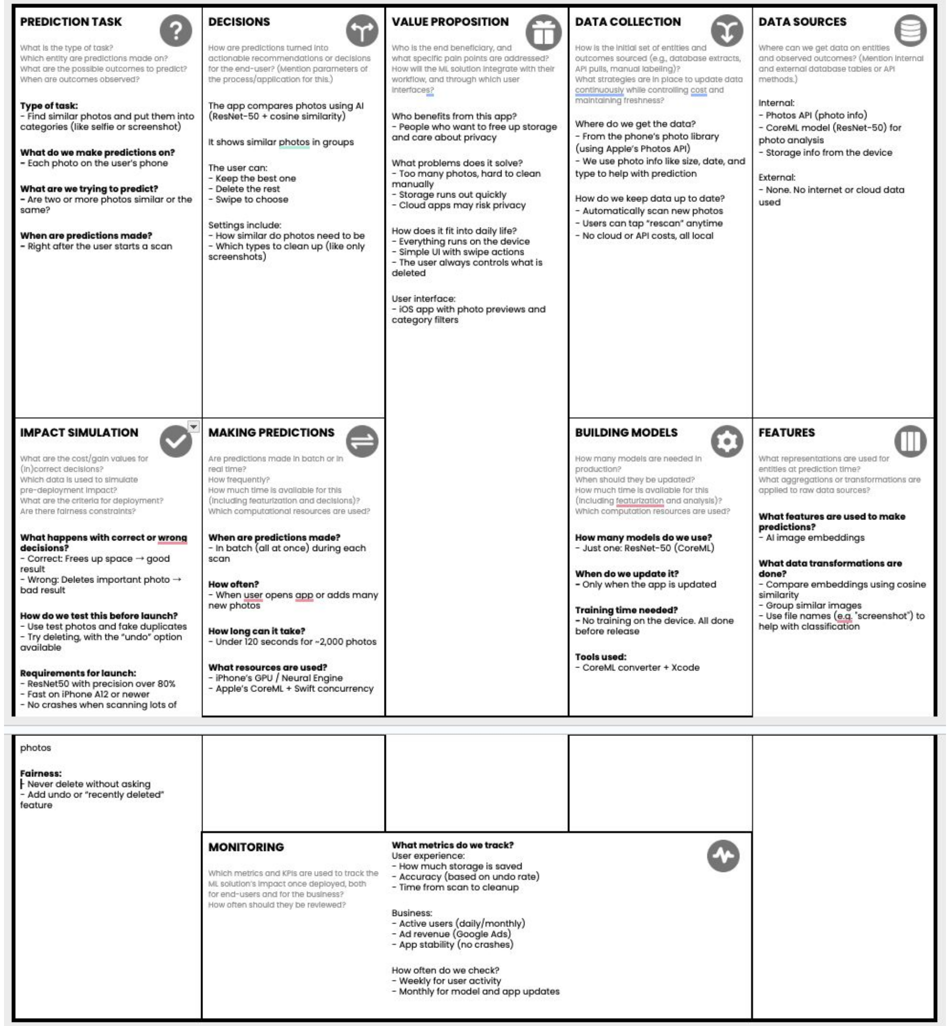
- Existing apps for photo cleaning usually require cloud upload or charge a subscription fee.
- Popular competitors (like Cleanup Pro, Cleaner AI) cost \$29.99–\$39.99 and process data in the cloud.
- Drawbacks:
  - Privacy risk: Photos are uploaded to servers
  - Cost: Users must pay for premium features
  - Extra functions: Apps bundle many features users may not need

<u>Cleaner AI Premium Lifetime \$39.99</u>
All-in-one app removing duplicates photo, merging contacts, compressing videos, delete emails, and protecting private files.

<u>Cleanup Pro 1 Year \$29.99</u>
Remove duplicate photos, videos, and junk emails. Merge duplicate contacts, compress videos, and organize storage

ML Canvas

- Prediction Task:
  - Detect similar photos by compare embeddings using cosine similarity
- Data & Model:
  - Data from the phone’s photo library
  - Runs [CoreML Pretrained ResNet-50] (MIT License) for image feature extraction directly on the device
- Value Proposition:
  - Saves phone storage by removing duplicate or unwanted photos.
  - No photos are ever uploaded to the cloud which the privacy is protected!
- Decisions & User Actions:
  - The app shows groups of similar images.Users can keep the best photo and delete the rest with a single tap or swipe.



PREDICTION TASK

What is the type of task?
Which entity are predictions made on?
What are the possible outcomes to predict?
When are outcomes observed?

Type of task:
- Find similar photos and put them into categories (like selfie or screenshot)

What do we make predictions on?
- Each photo on the user’s phone

What are we trying to predict?
- Are two or more photos similar or the same?

When are predictions made?
- Right after the user starts a scan

DECISIONS

How are predictions turned into actionable recommendations or decisions for the end-user? (Mention parameters of the process/application for this.)

The app compares photos using AI (ResNet-50 + cosine similarity)

It shows similar photos in groups

The user can:
- Keep the best one
- Delete the rest
- Swipe to choose

Settings include:
- How similar do photos need to be
- Which types to clean up (like only screenshots)

VALUE PROPOSITION

Who is the end beneficiary, and what specific pain points are addressed? How will the ML solution integrate with their workflow, and through which user interfaces?

Who benefits from this app?
- People who want to free up storage and care about privacy

What problems does it solve?
- Too many photos, hard to clean manually
- Storage runs out quickly
- Cloud apps may risk privacy

How does it fit into daily life?
- Everything runs on the device
- Simple UI with swipe actions
- The user always controls what is deleted

User interface:
- iOS app with photo previews and category filters

DATA COLLECTION

How is the initial set of entities and outcomes sourced (e.g., database extracts, API pulls, manual labeling)?
What strategies are in place to update data continuously while controlling cost and maintaining freshness?

Where do we get the data?
- From the phone’s photo library (using Apple’s Photos API)
- We use photo info like size, date, and type to help with prediction

How do we keep data up to date?
- Automatically scan new photos
- Users can tap “rescan” anytime
- No cloud or API costs, all local

DATA SOURCES

Where can we get data on entities and observed outcomes? (Mention internal and external database tables or API methods.)

Internal:
- Photos API (photo info)
- CoreML model (ResNet-50) for photo analysis
- Storage info from the device

External:
- None. No internet or cloud data used

IMPACT SIMULATION

What are the cost/gain values for (in)correct decisions?
Which data is used to simulate pre-deployment impact?
What are the criteria for deployment?
Are there fairness constraints?

What happens with correct or wrong decisions?
- Correct: Frees up space → good result
- Wrong: Deletes important photo → bad result

How do we test this before launch?
- Use test photos and fake duplicates
- Try deleting, with the “undo” option available

Requirements for launch:
- ResNet50 with precision over 80%
- Fast on iPhone A12 or newer
- No crashes when scanning lots of photos

Fairness:
- Never delete without asking
- Add undo or “recently deleted” feature

MAKING PREDICTIONS

Are predictions made in batch or in real time?
How frequently?
How much time is available for this (including featurization and decisions)?
Which computational resources are used?

When are predictions made?
- In batch (all at once) during each scan

How often?
- When user opens app or adds many new photos

How long can it take?
- Under 120 seconds for ~2,000 photos

What resources are used?
- iPhone’s GPU / Neural Engine
- Apple’s CoreML + Swift concurrency

BUILDING MODELS

How many models are needed in production?
When should they be updated?
How much time is available for this (including featurization and analysis)?
Which computation resources are used?

How many models do we use?
- Just one: ResNet-50 (CoreML)

When do we update it?
- Only when the app is updated

Training time needed?
- No training on the device. All done before release

Tools used:
- CoreML converter + Xcode

FEATURES

What representations are used for entities at prediction time?
What aggregations or transformations are applied to raw data sources?

What features are used to make predictions?
- AI image embeddings

What data transformations are done?
- Compare embeddings using cosine similarity
- Group similar images
- Use file names (e.g. "screenshot") to help with classification

MONITORING

Which metrics and KPIs are used to track the ML solution’s impact once deployed, both for end-users and for the business?
How often should they be reviewed?

What metrics do we track?
User experience:
- How much storage is saved
- Accuracy (based on undo rate)
- Time from scan to cleanup

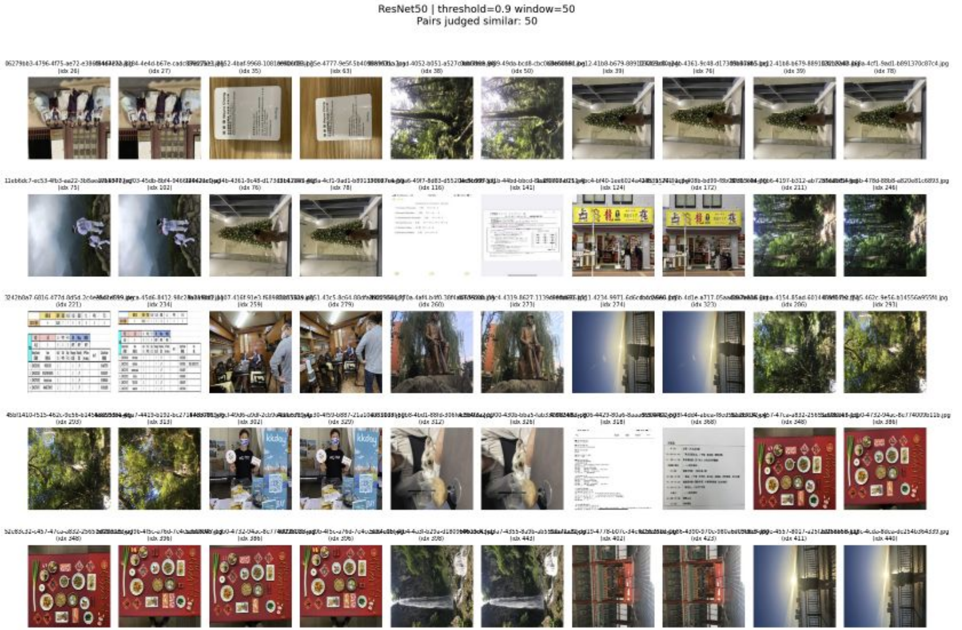
Business:
- Active users (daily/monthly)
- Ad revenue (Google Ads)
- App stability (no crashes)

How often do we check?
- Weekly for user activity
- Monthly for model and app updates

Model Benchmarking (ResNet & VIT Model)

Colab

	Architecture	Parameters	Embedding Dimension	Number of Images	Similar Pair Count	Description
ResNet50	ResNet50	25,600,000	2048	1100	65	Classic convolutional network, widely used for visual feature extraction.
CLIP_L/14	CLIP_L/14	428,000,000	768	1100	112	OpenAI's Vision Transformer, large patch size, text-image contrastive training.
CLIP_g/14-336	CLIP_g/14-336	1,300,000,000	768	1100	179	Largest CLIP ViT model, higher input resolution for fine-grained details.
OpenCLIP_H/14	OpenCLIP_H/14	633,000,000	1024	1100	77	OpenCLIP ViT-H/14, trained on LAION-2B dataset, 633M parameters.
DINOv2_B/14	DINOv2_B/14	86,000,000	768	1100	51	Self-supervised ViT from Meta, strong for unsupervised semantic grouping.
SAM_ViT-H/14	SAM_ViT-H/14	632,000,000	256	1100	23339	Meta AI's Segment Anything image encoder (ViT-H), massive context window.

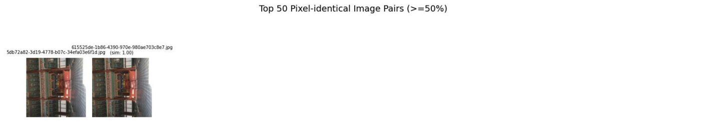


Model Benchmarking (Traditional Way)

1. Pixel Similarity

`Pixel Similarity > 0.5 pairs: 1`

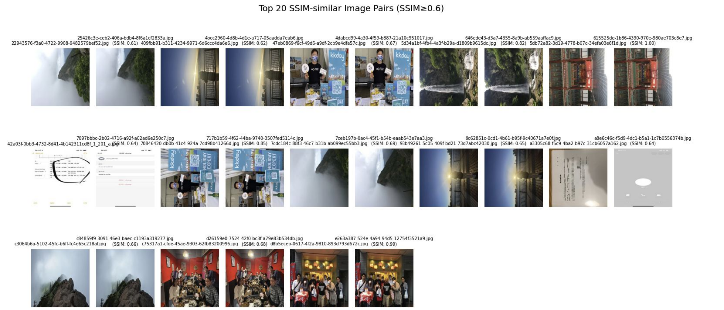
- **How it works:**
  Compares each pixel of two images directly.
- **Result:**
  Calculates what percentage of pixels are exactly the same.
- **Use case:**
  Good for detecting duplicates, but sensitive to small changes (like brightness, resizing, or cropping).



2. Structural Similarity Index (SSIM)

`SSIM > 0.6 pairs: 13`

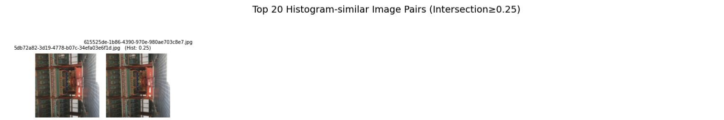
- **How it works:**
  Looks at overall structure, brightness, and contrast rather than just pixel values.
- **Result:**
  Gives a score from 0 (different) to 1 (very similar).
- **Use case:**
  Great for finding images that “look alike” even if there are small edits or noise.



3. Histogram Intersection

Histogram intersection > 0.25 pairs: 1

- **How it works:**
  Compares the distribution of colors (histograms) between images.
- **Result:**
  Higher value means similar overall color composition.
- **Use case:**
  Useful for finding images with similar color patterns, but may miss structural differences.



Model Deployment in Smart AI Photo Cleaner APP

On-Device Model Storage:

- The AI model (ResNet-50) is saved directly on the user's smartphone. (100 MB)

Local Execution:

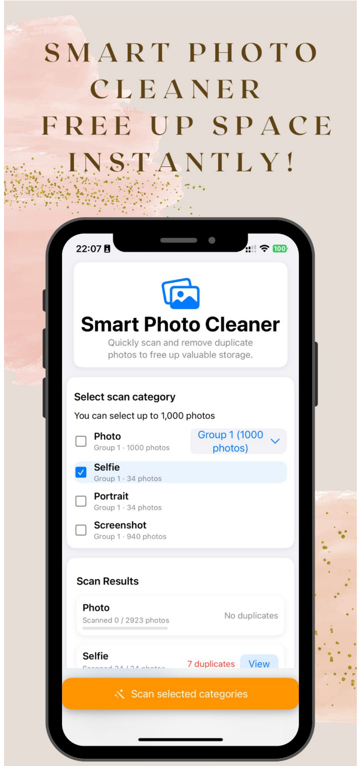
- All image scanning, similarity calculation, and photo classification are done locally, with no data sent to any server.

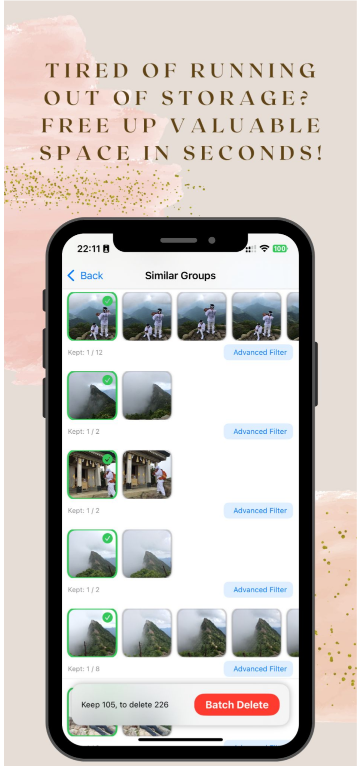
No Cloud Required:

- No need for cloud APIs, web servers, or external processing—*everything works offline*.

Key Benefits:

- **Fast**: Instant results without network delay
- **Private**: User photos never leave the device
- **Secure**: No risk of data leaks





Smart AI Photo Cleaner APP

Q & A

Github